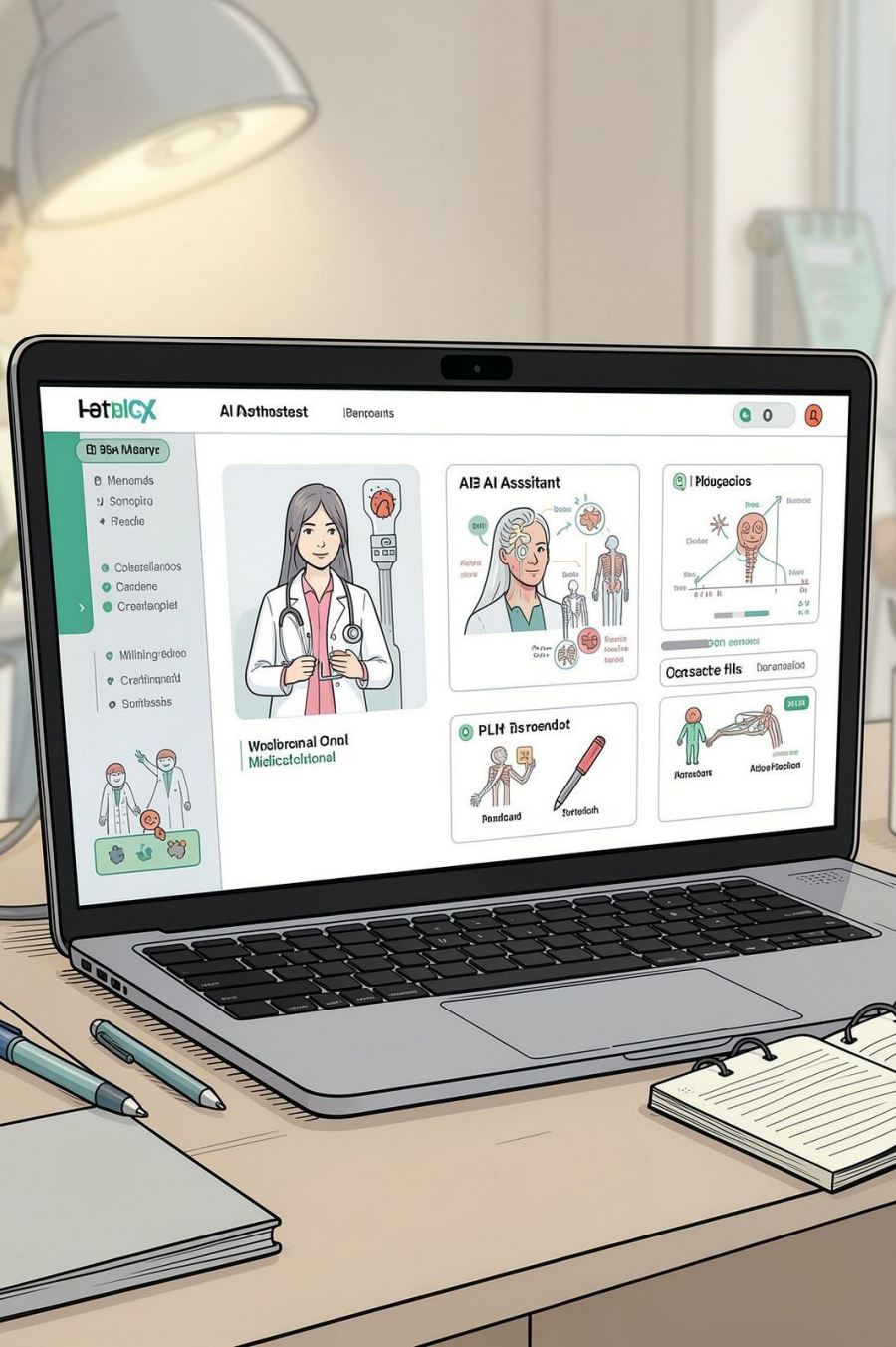




MediGuide AI — Educational Medical AI Prototype

A safety-first Streamlit demo that turns patient-reported symptoms into structured, educational guidance: a concise summary, possible conditions (for education only), an urgency level, and suggested questions to ask a clinician.

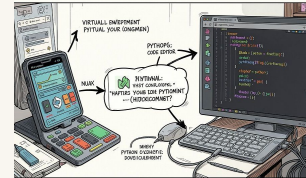
Quick Access



Live demo

Visit the deployed prototype to test inputs and observe live streaming responses.

<https://medical-ai-assistant-1abdullah.streamlit.app>

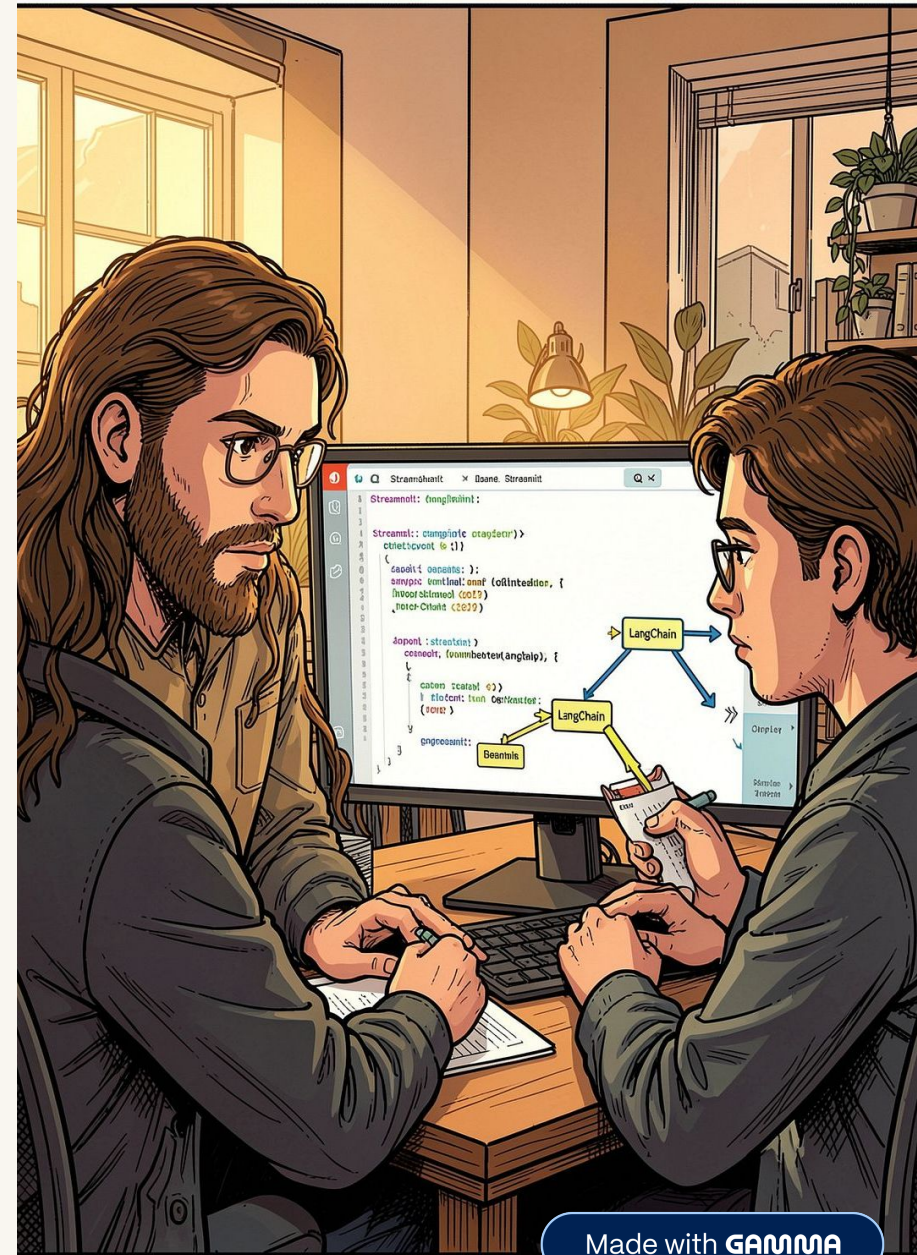


Run locally

Clone repo, create venv, install requirements, copy `.env.example` to `.env` and add `OPENAI_API_KEY`, then run `streamlit run app.py`.

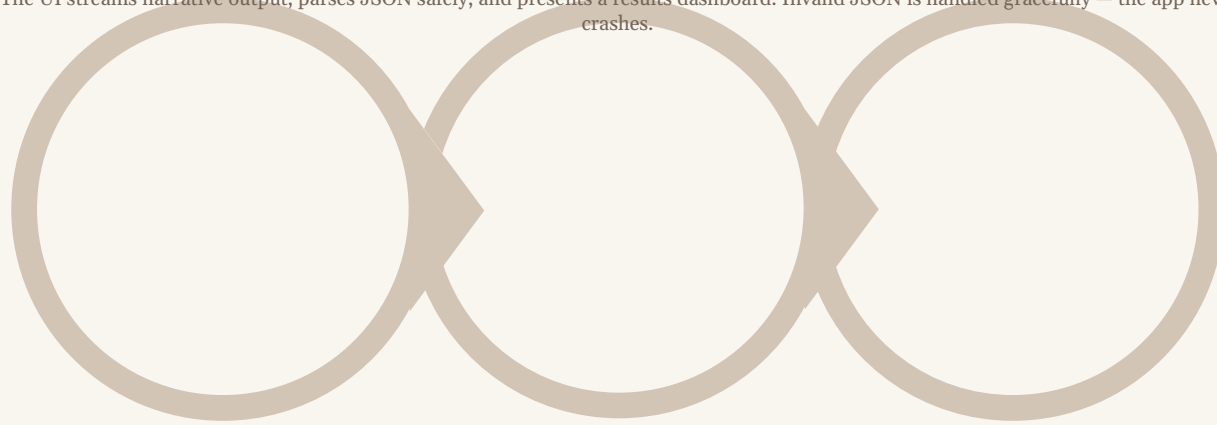
Project Structure — At a Glance

Key files: `src/config.py` — settings & env handling; `src/prompts.py` — safety-focused prompt templates + JSON schema; `src/chains.py` — LangChain LLM chains and streaming; `src/cache_manager.py` — InMemory and SQLite caching; `src/utils.py` — resilient JSON parsing and helpers; `app.py` — Streamlit UI glue.



How It Works — Data Flow

The UI streams narrative output, parses JSON safely, and presents a results dashboard. Invalid JSON is handled gracefully — the app never crashes.



Safety & Limitations (Non-negotiable)

❏ Prototype for education only — not a medical device or a replacement for clinical judgement.

Never provide diagnoses

The system must not present confirmed diagnoses or definitive clinical decisions.

Clear disclaimers

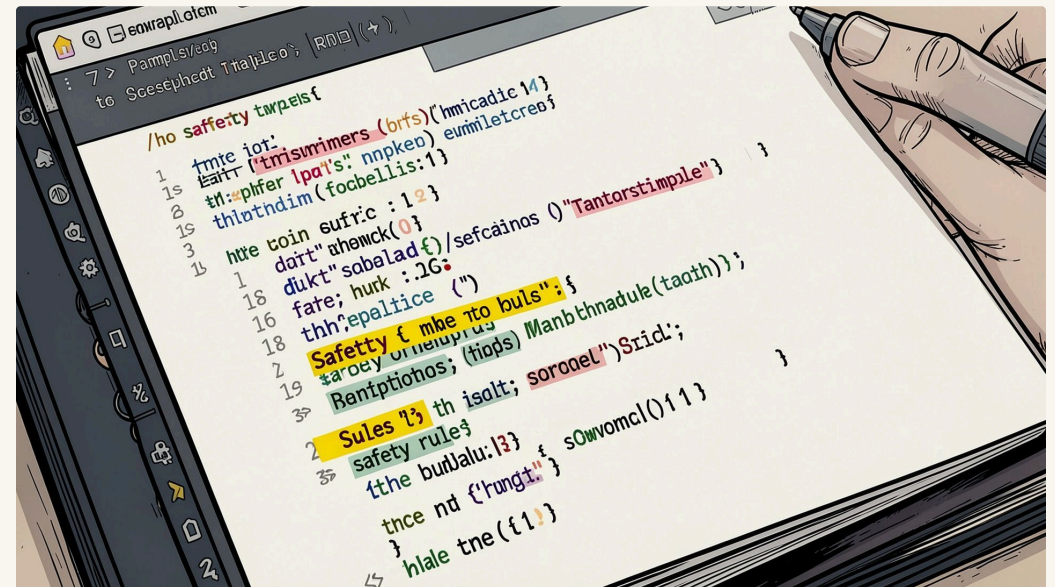
Persistent safety notice in the sidebar; emergency instructions displayed for high-urgency cases.

JSON-only response

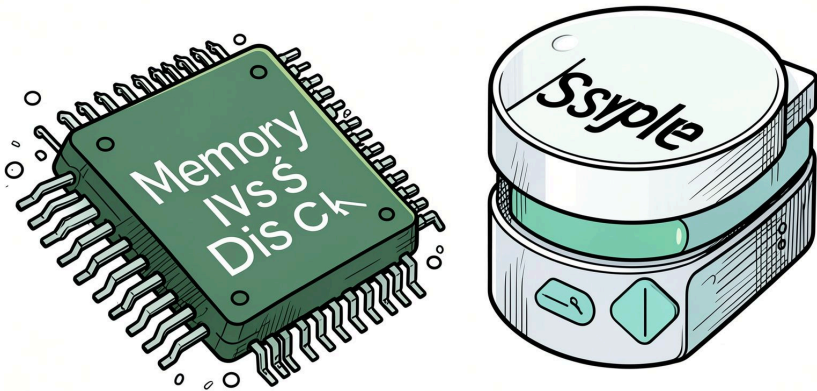
Model is instructed to return valid JSON (no markdown, no extra explanation) for safe programmatic parsing.

Prompting & Robustness Improvements

Corrections applied in src/prompts.py: merged duplicate JSON_SCHEMA_INSTRUCTION, added strict safety rules, replaced placeholder demo text with real patient fields ({age},{gender},{symptoms}, etc.), and ensured HumanMessagePromptTemplate uses patient-data template.



Caching Strategies — Why they matter



Switchable from the sidebar; test caching by submitting the same form twice to observe speed difference.



InMemoryCache

Fastest; session-only; clears on restart — ideal for development and rapid iteration.



SQLiteCache

Persistent across restarts; slightly slower; ideal for reproducible responses and cost savings.

Key Testing Scenarios

1

Common mild illness

Age 25, runny nose & sore throat → Expected: LOW urgency, self-monitoring advice.

2

Moderate symptoms

Age 40, fever & cough 4–7 days → Expected: MEDIUM/HIGH urgency, recommend clinician visit.

3

Emergency red flags

Severe chest pain or shortness of breath → Expected: HIGH/EMERGENCY, instruct to call emergency services immediately.

4

Empty input handling

Empty symptoms → App warns and avoids API call.

5

Localization

Language selection (e.g., Urdu) returns guidance in that language when configured in sidebar.

Development Notes & Bugfix Log

Notable fixes during development: - config.py: ensured OPENAI_API_KEY read via os.getenv; - prompts.py: fixed duplicate variables and prompt formatting errors; - chains.py: removed stray calls and passed the actual API key variable; - .env location: moved to project root to avoid missing-credentials errors; - regenerated compromised API key after accidental paste.



Next Steps & Responsible Roadmap

Immediate

- Security audit of logging and secrets
- Expand prompt tests and schema validation
- User testing with clinicians for wording & triage thresholds

Future

- Formal risk assessment for any clinical use
- Federated or encrypted caching for PHI avoidance
- Integration of evidence summaries & citation prompts for transparency

 Remember: Always advise users to consult a qualified healthcare professional. In emergencies, contact local emergency services.

Repo & run instructions: clone, install, add OPENAI_API_KEY to .env, then streamlit run app.py. Redeploy and test only after review.